

Resolução do Exame Modelo 2 — Paradigmas de Programação (2025/2026)

Parte 1 — Teoria (6 Valores)

Pergunta 1: Encapsulamento e Modificadores de Acesso (1,5 valores)

Resposta: O **encapsulamento** oculta os detalhes de implementação e a estrutura interna de dados de uma classe, exposta apenas através de métodos públicos controlados. O modificador `private` limita o acesso à própria classe; `protected` permite acesso no mesmo pacote e por subclasses em qualquer pacote; `public` permite acesso universal. O acesso direto aos atributos é uma má prática pois permite que o estado interno do objeto seja modificado arbitrariamente para valores inválidos sem qualquer controlo.

```
public class ContainerValidador {
    private double capacidade;

    public double getCapacidade() { return this.capacidade; }

    public void setCapacidade(double capacidade) {
        if (capacidade > 0) { // Validação defensiva
            this.capacidade = capacidade;
        }
    }
}
```

Pergunta 2: Polimorfismo (Overloading vs Overriding) (1,5 valores)

Resposta: **Sobrecarga (Overloading)** ocorre na mesma classe com métodos de igual nome mas assinaturas distintas (parâmetros diferentes). A resolução de qual método invocar é feita pelo compilador em *compile-time*.

Sobreposição (Overriding) ocorre em herança/interfaces quando uma subclasse redefine um método herdado mantendo a mesma assinatura. A resolução é feita pela JVM em *runtime* através do tipo real da instância alocada na Heap (ligação dinâmica / *dynamic binding*).

Pergunta 3: Tratamento de Exceções (1,5 valores)

Resposta: **Checked Exceptions** (subclasses de `Exception`) representam condições imprevisíveis de negócio que o compilador obriga a tratar (`try-catch`) ou a declarar na assinatura (`throws`). **Unchecked Exceptions** (`RuntimeException`) representam erros de programação e não exigem declaração. O bloco `finally` é executado SEMPRE para libertar recursos. Exceções personalizadas (ex: `AidBoxException`) devem ser criadas para encapsular erros específicos do domínio.

Pergunta 4: Membros `static` e Modificador `final` (1,5 valores)

Resposta: Membros `static` pertencem à classe (existe uma única cópia partilhada na área de classes). O modificador `final` impede a reatribuição de variáveis, sobreposição de métodos e herança de classes. Num tipo primitivo `final`, o valor numérico não muda. Numa referência `final`, o endereço de memória é imutável, mas o estado interno do objeto apontado pode ser alterado.

Parte 2 — Prática (14 Valores)

Pergunta 1a: Classe `AidBoxImpl` (3 valores)

```
public class AidBoxImpl implements AidBox {
    private String code;
    private String zone;
    private Container[] containers;
    private int count;

    public AidBoxImpl(String code, String zone) {
        this.code = code;
        this.zone = zone;
        this.containers = new Container[4]; // Capacidade maxima de 4
        this.count = 0;
    }

    @Override public String getCode() { return this.code; }
    @Override public String getZone() { return this.zone; }

    @Override
    public Container[] getContainers() {
        Container[] result = new Container[this.count];
        System.arraycopy(this.containers, 0, result, 0, this.count);
        return result;
    }

    public void addContainer(Container container) throws AidBoxException {
        if (container == null) throw new AidBoxException("Container nulo!");
        if (this.count >= 4) throw new AidBoxException("Capacidade maxima atingida!");
        this.containers[this.count] = container;
        this.count++;
    }

    @Override
    public boolean equals(Object obj) {
        if (this == obj) return true;
        if (obj == null || !(obj instanceof AidBox)) return false;
        AidBox other = (AidBox) obj;
        if (this.code == null) return other.getCode() == null;
        return this.code.equals(other.getCode());
    }
}
```

Pergunta 1b: Método `main` de Teste (2 valores)

```
public class MainTestAidBox {
    public static void main(String[] args) {
        System.out.println("=== TESTE DA CLASSE AidBoxImpl ===");

        AidBoxImpl box1 = new AidBoxImpl("BOX01", "Porto Centro");
        System.out.println("Codigo: " + box1.getCode());
        System.out.println("Zona: " + box1.getZone());

        try {
            box1.addContainer(new ContainerImpl("C01", ItemType.PERISHABLE_FOOD, 100.0));
            box1.addContainer(new ContainerImpl("C02", ItemType.CLOTHING, 200.0));
            System.out.println("Contentores adicionados com sucesso.");
        } catch (AidBoxException e) {
            System.out.println("Erro: " + e.getMessage());
        }

        AidBoxImpl box2 = new AidBoxImpl("BOX01", "Outra Zona");
        System.out.println("box1.equals(box2): " + box1.equals(box2)); // true
    }
}
```

Pergunta 2a e 2b: Métodos de Relatório em `ReportImpl` (9 valores)

```
public class ReportImpl implements Report {

    public int countContainersByType(AidBox aidbox, ItemType type) {
        if (aidbox == null || type == null) return 0;
        Container[] containers = aidbox.getContainers();
        if (containers == null) return 0;

        int count = 0;
        for (int i = 0; i < containers.length; i++) {
            if (containers[i] != null && containers[i].getType() == type) {
                count++;
            }
        }
        return count;
    }

    public double getAverageOccupancy(AidBox aidbox) {
        if (aidbox == null) return 0.0;
        Container[] containers = aidbox.getContainers();
        if (containers == null || containers.length == 0) return 0.0;

        double sumPercent = 0.0;
        int validContainers = 0;

        for (int i = 0; i < containers.length; i++) {
            Container c = containers[i];
            if (c != null && c.getLastMeasurement() != null && c.getCapacity() > 0) {
                double occ = (c.getLastMeasurement().getValue() / c.getCapacity()) * 100.0;
                sumPercent += occ;
            }
        }
    }
}
```

```

        validContainers++;
    }
}

if (validContainers == 0) return 0.0;
return sumPercent / validContainers;
}

@Override
public String generate(IIInstitution inst) {
    if (inst == null) return "";
    AidBox[] boxes = inst.getAidBoxes();
    if (boxes == null) return "";

    String report = "=== RELATORIO DE AIDBOXES (>50% OCUPACAO) ===\n";

    for (int i = 0; i < boxes.length; i++) {
        AidBox box = boxes[i];
        if (box != null) {
            double avg = getAverageOccupancy(box);
            if (avg > 50.0) {
                report += "Code: " + box.getCode() + " | Zona: " + box.getZone() + "\n";
                report += "  Ocupacao Media: " + avg + "%\n";
                report += "  Food Perecivel: " + countContainersByType(box, ItemType.PERISHABLE) + "\n";
                report += "  Food Nao Perecivel: " + countContainersByType(box, ItemType.NON_PERISHABLE) + "\n";
                report += "  Vestuario: " + countContainersByType(box, ItemType.CLOTHING) + "\n";
                report += "  Medicamentos: " + countContainersByType(box, ItemType.MEDICINE) + "\n";
                report += "-----\n";
            }
        }
    }

    return report;
}
}

```